

# 강의교안 이용 안내

- 본 강의교안의 저작권은 도서출판 홍릉에 있습니다.
- 불법한 PDF 파일이 사용되지 않도록 지도 바랍니다.
- 이 자료를 무단으로 전제하거나 배포할 경우 저작권법 136조에 의거하여 벌금에 처할 수 있고 이를 병과(併科)할 수도 있습니다.

# 단원 10

## 예외 처리와 파일 입출력

인공지능소프트웨어학과

강환수 교수



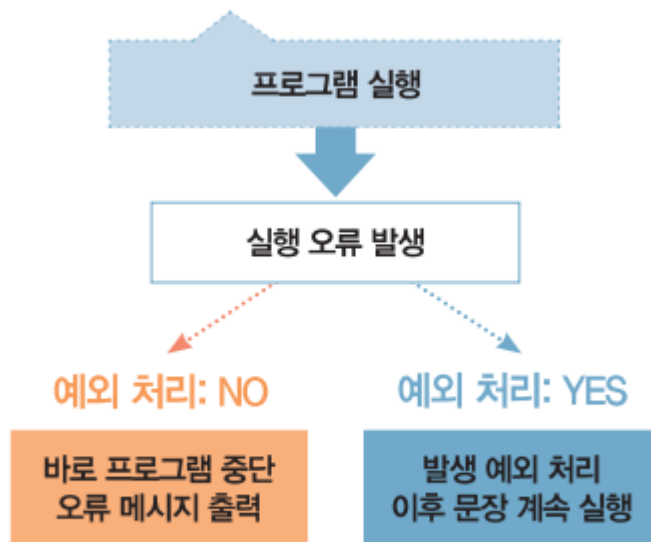
## Section 1. 실행 오류로 인한 프로그램 종단을 막는 예외 처리



## 예외 처리 개요

### • 예외(exception)

- 여러 원인에 의해 실행 중에 발생하는 오류
- 예외 처리(exception handle)
  - 발생한 예외를 처리하여 프로그램이 바로 종료되는 일을 막도록 하는 것



▲ 그림 1 실행 오류와 예외 처리

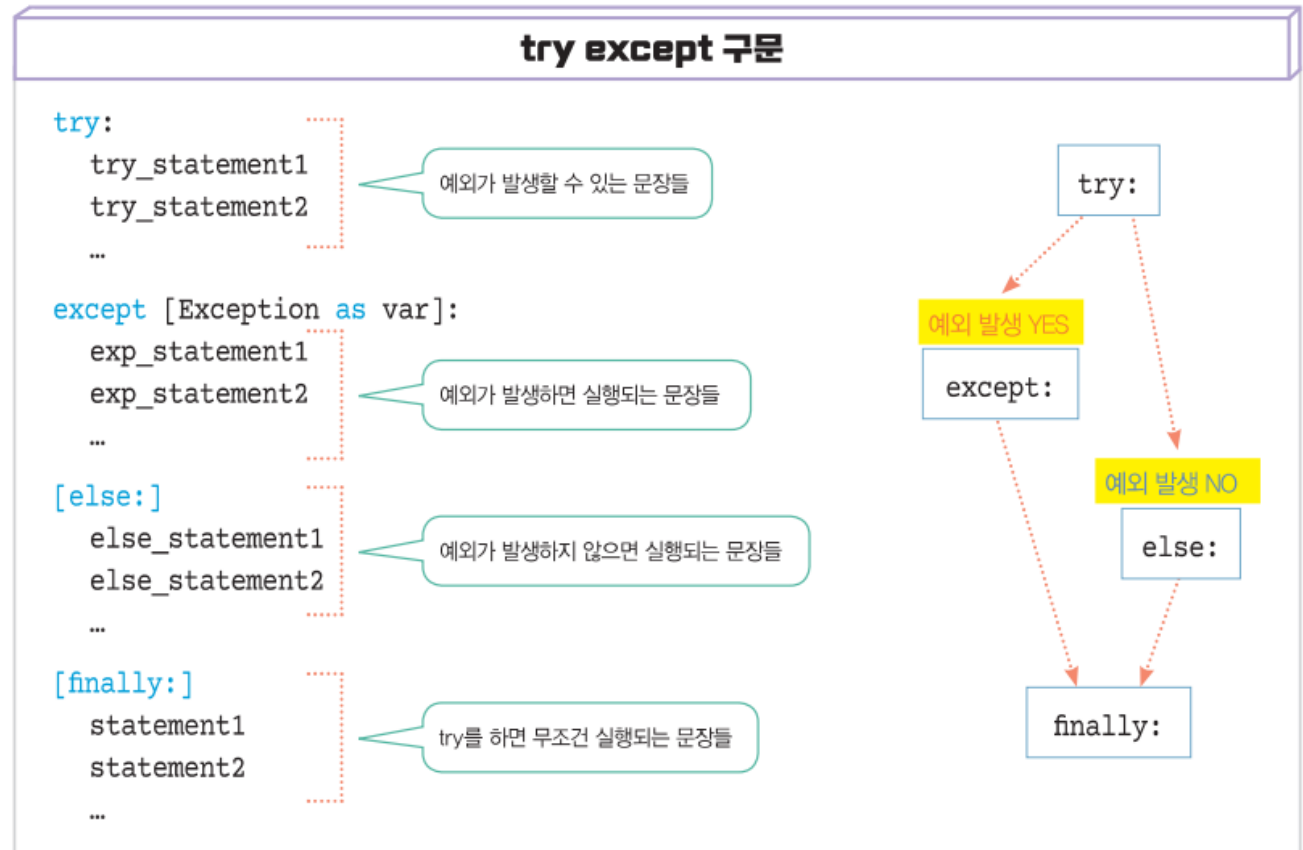
```
>>> 3.4 / 0
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: float division by zero
```

## 예외 처리 구문 try except

예외 처리는 try except 구문으로 실행

- 예외 발생이 의심스러운 문장들을 try 블록에 배치
  - 예외가 있을 경우 실행될 코드를 except 블록에 배치
    - try 블록의 문장들에서 예외가 발생하면 except 블록이 실행
  - 옵션 else: 블록
    - 예외가 발생하지 않으면 실행
  - 옵션 finally: 블록
    - 예외 발생과 상관없이 try 블록을 실행하면 반드시 실행



## 예외가 발생한 정보를 저장하여 처리하는 except as

- **except** 이후에 '발생된\_예외객체\_클래스이름 as 변수명'을 추가

- 예외가 발생하면 예외 인스턴스(instance)라는 것이 발생
- 이 객체를 변수 e로 받아 예외 상황에 대한 정보를 활용

```
>>> try:
...     10 / 0
... except Exception as e:
...     print(f'예외 발생 유형: {type(e)}')
...     print(f'예외 발생 이유: {e}')
...
```

예외 발생 유형: <class 'ZeroDivisionError'>

예외 발생 이유: division by zero

발생된 예외의 클래스의 이름을 알 수 있다.

발생한 예외의 원인을 알 수 있다.

# 제일 간단한 예외 처리

```
try:
```

```
    10 / 0
```

```
except:
```

```
    print('예외가 발생했습니다.')
```

## 예외와 관련된 클래스

객체지향 언어는 모든 예외를 클래스로 정의하여 관리

### • 클래스 `BaseException`

- 예외의 가장 상위 클래스
- 우리가 접하는 많은 예외 클래스는 `Exception` 클래스의 하위 클래스

### • `IndexError` 예외

- 리스트나 튜플 참조에서 첨자 범위를 벗어나면 발생

```
>>> try:
...     print([1, 2, 3][4])
... except IndexError as e:
...     print(f'예외 발생 유형: {type(e)}')
...     print(f'예외 발생 이유: {e}')
...
예외 발생 유형: <class 'IndexError'>
예외 발생 이유: list index out of range
```

```
>>> BaseException.__subclasses__()
[<class 'BaseExceptionGroup'>, <class 'Exception'>,
<class 'GeneratorExit'>, <class 'KeyboardInterrupt'>,
, <class 'SystemExit'>]
```

```
BaseException
+-- SystemExit
+-- KeyboardInterrupt
+-- GeneratorExit
+-- Exception
    +-- StopIteration
    +-- StopAsyncIteration
    +-- ArithmeticError
        |   +-- FloatingPointError
        |   +-- OverflowError
        |   +-- ZeroDivisionError
    +-- AssertionError
    +-- AttributeError
    +-- BufferError
    +-- EOFError
    +-- ImportError
        |   +-- ModuleNotFoundError
    +-- LookupError
        |   +-- IndexError
        |   +-- KeyError
    +-- MemoryError
    +-- NameError
        |   +-- UnboundLocalError
    +-- OSError
        |   +-- BlockingIOError
        |   +-- ChildProcessError
        |   +-- ConnectionError
            |   +-- BrokenPipeError
            |   +-- ConnectionAbortedError
            |   +-- ConnectionRefusedError
            |   +-- ConnectionResetError
    +-- FileExistsError
    +-- FileNotFoundError
    +-- InterruptedError
    +-- IsADirectoryError
    +-- NotADirectoryError
    +-- PermissionError
    +-- ProcessLookupError
    +-- TimeoutError
```

## 예외 처리 구문 try except else finally

- 문자열 첨자가 범위를 벗어나 잘못된 문자열 참조 예외 IndexError가 발생
  - except와 finally 블록이 실행

교재 p471에서 'python'으로 수정

```
try:
    print('pyhton'[6])
except IndexError as e:
    print('예외 발생 이름: {}'.format(type(e)))
    print('예외 발생 이유: {}'.format(e))
else:
    print('예외 없이 잘 실행되었습니다.')
finally:
    print('예외 처리가 잘되는군요!')
```

### 실행 결과

예외 발생 이름: <class 'IndexError'>  
예외 발생 이유: string index out of range  
예외 처리가 잘되는군요!

## try에서 여러 예외 처리 구현

### • 여러 개의 except: 블록

- try 블록에서 예상되는 예외 처리가 다양
- 예외가 발생하면 많아야 하나의 except: 블록만 실행
- 발생한 예외가 적시한 예외 클래스가 동일하거나 하부 클래스가 아니라면
  - except 블록은 실행되지 않음

```
try:
    x = int(input('정수 입력 > '))
    y = int(input('정수 입력 > '))
    print(f'{x} × {y} = {x * y}')
    print(f'{x} ÷ {y} = {x / y}')
except ZeroDivisionError as exp:

    print('except ZeroDivisionError as exp:')
    print(f'\t예외 발생 이유: {exp}')

except ValueError as exp:
    print('except ValueError as exp:')
    print(f'\t예외 발생 이유: {exp}')
```

실행 결과 1: 두 번째 입력에 0을 입력하는 경우

정수 입력 > 10

정수 입력 > 0

10 × 0 = 0

except ZeroDivisionError as exp:

예외 발생 이유: division by zero

실행 결과 2: 정수가 아닌 것을 입력하는 경우

정수 입력 > 3.2

except ValueError as exp:

예외 발생 이유: invalid literal

for int() with base 10: '3.2'

## 문장 raise를 사용하여 직접 예외를 발생

- 음수나 0이 입력되면 raise Exception('필요한 메시지') 구문으로 예외를 직접 발생

```
n = int(input('양의 정수 입력 > '))
if n <= 0:
    raise Exception('양의 정수를 입력하십시오.')
```

```
# %% 10-04raiseexception.py 직접 예외를 발생시켜 예외 처리
import math
```

```
try:
    n = int(input('양의 정수 입력 > '))
    if n <= 0:
        raise Exception('양의 정수를 입력하세요.')
```

```
except ValueError as exp:
    print(f'예외발생 유형: {type(exp)}')
    print(f'예외발생 이유: {exp}')
    print('입력한 값이 정수가 아닙니다.')
```

```
except Exception as exp:
    print(f'예외발생 유형: {type(exp)}')
    print(f'예외발생 이유: {exp}')
    print('입력한 값이 양의 정수가 아닙니다.')
```

```
else:
    print(f'{n}! = {math.factorial(n)}')
```

```
finally:
    print(' 프로그램 종료! '.center(30, '*'))
```

## 조건 인증 구문과 assert와 예외 처리

구문 assert는 어떤 조건이 True임을 인증하기 위해서 사용하는 문장

- 조건을 만족하지 못하면 예외 **AssertionError**가 발생하고 종료
  - 구문 assert는 개발 중에 디버깅 및 테스트용 도구로 사용

### 문장 assert

```
assert condition, error_message
```

- condition: 조건이 True이면 문제 없이 이후 문장을 실행
- 만일 조건이 False이면 "AssertionError: error\_message" 예외가 발생

```
>>> import sys
>>> sys.platform
'win32'
>>> sys.version
'3.12.0 (tags/v3.12.0:0fb18b0, Oct  2 2023, 13:03:39) [MSC v.1935 64 bit (AMD64)]'
>>> assert sys.version[:4] == '3.12', "파이썬 버전 3.12에서만 가능합니다."
>>>
>>> pl = {}
>>> pl['java'] = 1995
>>> print(f'{pl['java']}')
1995
```

❖ 구문 assert 이후에는 python 버전이 3.12에서만 실행되도록

## assert 활용

- try 구문으로 예외 처리

```
>>> def check_age(age):
...     assert age >= 18, "나이는 18세 이상이어야 합니다."
...     return "성인"
...

>>> check_age(17)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in check_age
AssertionError: 나이는 18세 이상이어야 합니다.

>>> try:
...     check_age(18)
...     check_age(17)
... except Exception as e:
...     print(f'예외 발생 유형: {type(e)}')
...     print(f'예외 발생 이유: {e}')
...
'성인'
예외 발생 유형: <class 'AssertionError'>
예외 발생 이유: 나이는 18세 이상이어야 합니다.
```

## Section 2. 파일을 생성하고 내용을 읽는 파일 입출력



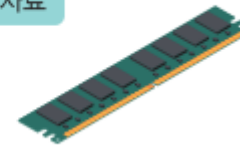
## 파일 개요

- 프로그램 실행 중에 저장한 변수
  - 메인 메모리인 RAM(Random Access Memory)에 저장
  - 프로그램이 종료되면 모두 사라지는 휘발성(volatile) 자료
- 영구 저장이 가능한 파일
  - 하드디스크나 USB 등의 보조 기억 장치에 저장

프로그램에서의 변수와 파일 사용

프로그램이 종료되면 다시 사용할 수 없는 자료

```
month = {1: 'January', 2: 'February', 3: 'March', 4: 'April'}
```



프로그램이 종료되더라도 다시 파일에서 읽어와 사용할 수 있는 자료

```
f = open('pyzen.txt', 'wt')
f.write('파이썬 철학\n')
f.write('아름다움이 추한 것보다 낫다.\n')
f.write('명시적인 것이 암묵적인 것보다 낫다.\n')
f.close()
```



▲ 그림 3 프로그램에서의 변수와 파일 사용

## 파일 쓰기 write()와 읽기 read()

- 내장 함수 open()
  - 파일 열기
- 함수 file.write('문자열')
  - 문자열을 파일에 쓰기
- 함수 read()
  - 파일의 모든 내용 전체를 읽는 함수
- 함수 close()
  - 파일 닫기

```
>>> file = open('python.txt', 'w')
>>> file.write('python file output')
18
>>> file.close()
```

모드는 쓰기를 의미하는 w이고 wt(write text)도 가능하다.

함수 write(문자열)로 문자열을 출력할 수 있다.

함수 write(문자열)는 출력한 문자열의 길이를 반환한다.

```
>>> file = open('python.txt', mode='r')
>>> print(file.read())
python file output
>>> file.close()
```

위에서 파일에 쓴 내용이 출력된다.

## 파일 열기 함수 open()

- 첫 두 인자는 file과 mode
  - 모드
    - 기본이 읽기 모드인 r로 파일에서 읽으려면 생략도 가능

### 내장 함수 open()

```
open( file, mode= 'r', ... )
```

- file: 참조하려는 파일의 이름이 들어 있는 문자열 값
- mode: 파일을 열어 처리하려는 모드로, 읽기(r), 쓰기(w), 추가(a) 등을 결정한다. 선택 매개 변수이며 기본 파일 액세스 모드는 읽기(r)이다.

## 텍스트 파일 열기 모드 종류

t 생략 가능, 파일에서 읽고, 쓰고, 추가하는 위치를 '파일 포인터'라 지칭

| 모드      | 모드                   | 설명                                                                |
|---------|----------------------|-------------------------------------------------------------------|
| r, rt   | 읽기 전용 파일의 열기         | 파일 내용 읽기 위치인 파일 포인터는 파일의 시작 부분에 위치하며, 지정하지 않으면 r이 기본 모드           |
| w, wt   | 쓰기 전용 파일의 열기         | 이미 파일이 있으면 파일을 덮어 쓰며, 파일이 없으면 쓰기 용으로 새 파일을 작성                     |
| a, at   | 추가할 파일의 열기           | 파일 추가 위치인 파일 포인터는 파일이 있는 경우 파일의 끝에 위치, 만일 파일이 없다면 쓰기 용으로 새 파일을 작성 |
| r+, rt+ | 읽기 및 쓰기 모드로 파일 열기    | 파일 내용을 읽거나 쓰는 위치인 파일 포인터는 파일의 시작 부분에 위치                           |
| w+, wt+ | 쓰기 및 읽기 모드의 파일 열기    | 파일이 있는 경우 기존 파일을 덮어쓰고, 파일이 없으면 읽기 및 쓰기 모드로 새 파일을 작성               |
| a+, at+ | 추가 및 읽기 모드의 파일 열기 모드 | 파일 포인터는 파일이 있는 경우 파일의 끝에 위치하며, 파일이 없으면 읽기 및 쓰기 모드로 새 파일 열기        |

## 파일 읽기 readlines()와 readline()

- 함수 readlines()

- 파일 전체 내용 한 줄 한 줄의 항목으로 구성된 리스트를 반환

- 함수 readline()

- 한 줄의 문자열을 읽어 반환하는 함수

```
# %% 10-06writereadfile.py 파일에 '파이썬 철학' 쓰기과 읽기
f = open('pyzen.txt', 'wt')
f.write('파이썬 철학\n')
f.write('아름다움이 추한 것보다 낫다.\n')
f.write('명시적인 것이 암묵적인 것보다 낫다.\n')
f.close()
```

```
>>> f = open('pyzen.txt', mode='r')
>>> lines = f.readlines()
>>> print(lines)
['파이썬 철학\n', '아름다움이 추한 것보다 낫다.\n', '명시적인 것이 암묵적인 것보다 낫다.\n']
>>> f.close()
```

```
f = open('pyzen.txt', 'r')

line = f.readline()
while line: # while line != '':
    print(line)
    line = f.readline()

f.close()
```

```
f = open('pyzen.txt', 'r')

while True:
    line = f.readline()
    if not line: break
    print(line)

f.close()
```

## 파일 읽기의 예외 처리

- 쓰기 모드인 'w'
  - open('filename', mode='w')
    - 이미 파일 filename이 있어도 기존의 내용을 모두 무시하고 덮어 씌
    - 물론 파일이 없다면 파일이름 filename으로 새로운 파일을 생성
- 읽기 모드인 'r'
  - 존재하지 않는 파일을 읽기 모드인 'r'로 open(... mode='r')
    - FileNotFoundError 예외가 발생

```
try:
    f = open('pyzen.txt', 'r')
except FileNotFoundError as e:
    print('예외 처리:', e)
else:
    print(f.read())
    f.close()
```

예외 발생 실행 결과: 파일 pyzen.txt가 없는 경우

예외 처리: [Errno 2] No such file or  
directory: 'pyzen.txt'

정상 실행 실행 결과: 파일 pyzen.txt가 있는 경우

파이썬 철학  
아름다움이 추한 것보다 낫다.  
명시적인 것이 암묵적인 것보다 낫다.

## with open(...) as file 구문

- close() 호출 없이 자동으로 파일을 닫아 줌

```
>>> with open('data.txt', mode='w') as file:  
...     file.write('파이썬')  
...  
3
```

```
>>> with open('data.txt', mode='r') as file:  
...     print(file.read())  
...  
파이썬
```

## open(file, mode='x') 사용

- mode='x'

- 파일 쓰기에서 기본 파일에 덮어쓰는 것을 방지
  - 모드 x는 배타적 생성(exclusive creation)이라고 하며, eXclusive 의 x를 사용
- 파일 열기의 mode='x'는 새로운 파일에만 기록
  - 이미 파일이 있다면 FileExistsError 예외가 발생

- mode='w'

- 파일이 없다면 새로 만들고,
- 이미 파일이 있어도 이전 내용을 무시하고 새로이 내용을 덮어써 이전 내용이 모두 사라짐

```
>>> with open('mode.txt', mode='x') as file:
...     file.write("mode='x'")
...
8
>>> with open('mode.txt', mode='x') as file:
...     file.write("mode='x'.....")
...
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
FileExistsError: [Errno 17] File exists: 'mode.txt'
```

위에서 이미 파일이 생성되어 FileExistsError  
예외가 발생한다.

## 함수 writelines()

- 함수 writelines()

- 여러 줄의 문자열을 한 번에 파일에 출력

```
lst = ['파일 처리 모드\n', "mode='x'\n", "mode='rt'\n"]  
with open('modex.txt', mode='x') as file:  
    file.writelines(lst)
```

- 반복적으로 한 줄 한 줄의 문자열을 line에 받아 처리가 가능

- file 사용

```
with open('modex.txt', mode='rt') as file:  
    for line in file:  
        print(line, end='')
```

```
# f.readlines() 활용도 가능  
F = open('pyzen.txt', 'r')  
For line in f.readlines():  
    print(line, end='')  
f.close()
```

## 바이너리 파일 쓰기

- 바이너리 파일(binary file)

- 이미지, 오디오, 비디오와 같은 텍스트가 아닌 파일

- 0 또는 1의 이진수 형태 그대로 저장된 파일이 바이너리 파일

- 정수나 실수, 리스트나 딕셔너리, 객체와 같은 파이썬의 다양한 자료형 그대로를 bytes로 변환해 파일로 저장

```
>>> value = [10, 20, 30]
>>> with open("value.bin", "wb") as f:
...     f.write(bytes(value))
...
3

>>> with open("value.bin", "rb") as f:
...     data = f.read()
...     print(data)
...     print(list(data))
...
b'\n\x14\x1e'
[10, 20, 30]
```

❖ 바이너리 파일에 자료형 그대로 쓰면 `TypeError` 발생

```
value = [10, 20, 30]
with open("value1.bin", "wb") as f:
    f.write(value)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[9], line 4
      1 # %%
      2 value = [10, 20, 30]
      3 with open("value1.bin", "wb") as f:
----> 4     f.write(value)

TypeError: a bytes-like object is required, not 'list'
```

## 표준 모듈 pickle 활용한 바이너리 파일 읽기와 쓰기

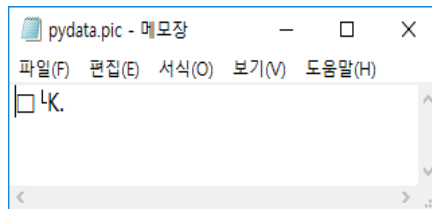
관례적으로(주로) 파일 확장자는 \*.pkl 또는 \*.pickle

### 파이썬 자료형 int 변수에 값 10을 저장하고 이 변수를 파일 pydata.pic에 저장

- 함수 pk.dump(obj, file): 객체 obj를 파일 file에 쓰기
- 함수 pk.load(file): 파일 file에서 쓰여 있는 객체를 읽어 반환

```
import pickle as pk
```

```
with open('pydata.pic', mode='wb') as f:
    # 파이썬 자료형 value를 파일 f에 쓰기
    value = 10
    pk.dump(value, f)
```



# 읽기: 파일 -> 객체

```
with open("pydata.pic", "rb") as f:
    loaded = pk.load(f)

print(loaded)
```

```
import pickle
```

```
my_list = [1, 2, 3]
my_dict = {"a": 10, "b": 20}
```

```
with open("data.pkl", "wb") as f:
    pickle.dump(my_list, f)
    pickle.dump(my_dict, f)
```

```
with open("data.pkl", "rb") as f:
    loaded_list = pickle.load(f)
    loaded_dict = pickle.load(f)
```

```
print(loaded_list)
print(loaded_dict)
```

## 바이너리 파일 열기 모드

- `open()`의 모드에서 바이너리를 의미하는 `b`를 붙여 사용

| 모드               | 설명                                                                                      |
|------------------|-----------------------------------------------------------------------------------------|
| <code>rb</code>  | 바이너리 형식으로 읽기 전용 파일의 열기 모드이며, 파일 포인터는 파일의 시작 부분에 위치시키고 기본 모드                             |
| <code>wb</code>  | 바이너리 형식으로 쓰기 전용 파일의 열기 모드이며, 이미 파일이 있으면 파일을 덮어 쓰며, 파일이 없으면 쓰기 용으로 새 파일을 작성              |
| <code>ab</code>  | 바이너리 형식으로 추가 할 파일의 열기 모드이며, 파일 포인터는 파일이 있는 경우 파일의 끝에 위치, 만일 파일이 없다면 쓰기 용으로 새 파일을 작성     |
| <code>rb+</code> | 바이너리 형식의 읽기 및 쓰기 위한 파일 열기로, 파일 포인터는 파일의 시작 부분에 위치                                       |
| <code>wb+</code> | 바이너리 형식의 쓰기 및 읽기를 위한 파일 열기로 파일이 있는 경우 기존 파일을 덮어 쓰고, 파일이 없으면 읽기 및 쓰기 모드로 새 파일을 작성        |
| <code>ab+</code> | 바이너리 형식의 추가 및 읽기를 위한 파일 열기로, 파일 포인터는 파일이 있는 경우 파일의 끝에 위치하며, 파일이 없으면 읽기 및 쓰기 모드로 새 파일 열기 |

## 파일 이름 수정 rename()과 삭제 remove()

- 표준 모듈인 os
  - 함수 rename(oldfilename, newfilename)
    - 이름 oldfilename을 newfilename으로 수정
  - 함수 remove(filename)
    - 파일 filename을 삭제

```
>>> import os
>>> os.rename('pyfile.txt', '파일수정삭제.txt') # 파일 이름 수정
>>> os.remove('파일수정삭제.txt') # 파일 삭제
```

## 다양한 폴더 관련 함수

### 표준 모듈 os

- 함수 `mkdir(dname)`
  - 폴더 `dname`을 생성
- 함수 `rmdir(dname)`
  - 폴더이름이 `dname`인 폴더를 삭제

```
>>> import os
>>> os.mkdir("python fold")
>>> os.rmdir("python fold")
```

| 함수                          | 설명              | 함수                                   | 설명                     |
|-----------------------------|-----------------|--------------------------------------|------------------------|
| <code>os.chdir(path)</code> | 현재 폴더를 path로 이동 | <code>os.rename(ofile, nfile)</code> | 파일 이름 ofile을 nfile로 수정 |
| <code>os.mkdir(d)</code>    | 폴더 d 생성         | <code>os.remove(file)</code>         | 파일 이름 file을 삭제         |
| <code>os.rmdir(d)</code>    | 폴더 d 삭제         | <code>os.path.exists(f)</code>       | f의 존재 여부 반환            |
| <code>os.getcwd()</code>    | 현재 폴더를 반환       | <code>os.path.isfile(f)</code>       | f의 파일 여부 반환            |
| <code>os.listdir(d)</code>  | 폴더 d 하부의 목록을 반환 | <code>os.path.isdir(f)</code>        | f의 폴더 여부 반환            |